Project Documentation

Design & Architecture Documentation

Project Title	Ease Travelling — Online Bus Ticket Booking System
Prepared By	Ishrat Fatima (71624); Muntaha Fatima (72092); Muqaddas Salam (70351); Muhammad Jawad Abid (70512); Rahimullah Rehmat (70751)
Department / Program	Software Engineering — BSIT2A
Supervisor / Instructor	Dr. Syed Muhammad Junaid Hassan

Prepared from the supplied project requirements and front-end prototype.

1. Introduction

1.1 Purpose

The purpose of this document is to explain the structure, main modules, interface design, data model, and booking workflow of Ease Travelling. It is intended for the project team, course instructor, and future developers who need to understand the design and the gap between the requirements and the current prototype.

1.2 Project Overview

Ease Travelling is a web-based online bus ticket booking system for passengers in Pakistan. The requirements describe searching buses by departure location, destination, and travel date; viewing schedules, prices, and seat availability; selecting seats; entering passenger details; completing a simulated payment; and receiving an electronic ticket.

1.4 Definitions and Abbreviations

Term	Meaning
UI	User Interface
UML	Unified Modeling Language
API	Application Programming Interface (if used)
SPA	Multiple -Page Application
[Other]	[Define project-specific terms]
Simulated payment	A demonstration payment flow; no real payment provider is connected.

2. System Design Overview

2.1 Design Goals

- Simple and user-friendly booking flow.
- Responsive layout for desktop and mobile.
- Clear display of routes, schedules, fares, and seat availability.
- Protect user and booking information.
- Maintainable code and understandable project structure.

2.2 Main System Modules

Implementation status note: The supplied index.html demonstrates a front-end prototype. UI elements and JavaScript interactions are present, but real accounts, durable bookings, shared seat inventory, server-side administration, and a persistent database are not evidenced. These must remain marked as planned until implemented and tested.

Module	Purpose	Actual status / notes
Authentication	Register, log in, log out, and update profile UI	Prototype only: handlers set a demo user in memory; no credential verification or persistent accounts.
Bus Search	Search by departure, destination, and date	Prototype interaction present; uses sample BUS_DATA and in-page state.

Bus Details	Show operator, route, schedule, fare, facilities, and seats	Bus cards and seat selection are represented in the front end; live data not connected.
Booking & Seats	Choose available seats and enter passenger information	Client-side demo state; cross-user seat locking is not implemented.
Payment	Simulate payment for the selected booking	Demo flow only; no real gateway or banking connection.
My Bookings / E-Ticket	Show booking records and ticket details	Ticket ID, QR code, and print styling are present; persistent booking history is not established.
Admin Panel	Manage buses, routes, schedules, users, seats, and bookings	Admin UI/navigation is present; persistent CRUD and authorization are not demonstrated.

3. Software Architecture

3.1 Architecture Style

The supplied implementation is best described as a client-side single-page web prototype. The browser renders the interface and runs JavaScript navigation and booking interactions. It uses an in-memory state object and sample bus data; no backend API calls, database connection, localStorage, or sessionStorage were found in the supplied HTML. A production release would need a server-side application and persistent database, but those are proposed future components rather than existing components.

3.2 Architecture Diagram

The diagram below separates the current browser-based prototype from backend/database components that are not present in the supplied HTML.

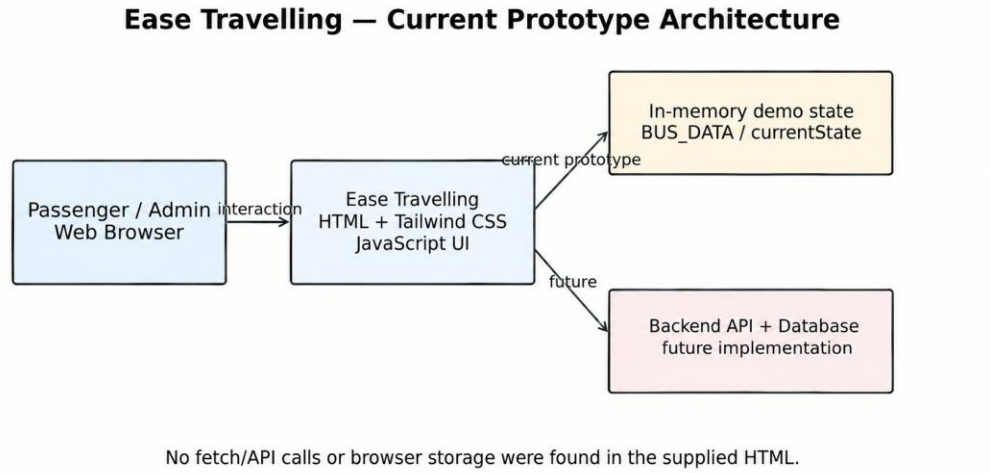


Figure 1. Current client-side prototype and proposed future backend boundary.

Figure 1. Overall architecture of Ease Travelling.

3.3 Component Responsibilities

Component / layer	Responsibility	Technology used
Frontend / UI	Single-page views, forms, bus cards, seat grid, ticket/QR display, navigation, responsive styling.	HTML5, Tailwind CSS CDN, custom CSS, JavaScript, Font Awesome 6.4.0, Google Fonts Inter, QRCode.js.
Backend / application	Business rules, authentication, booking validation, role checks, and payment	

	verification are not connected in the supplied files.	
Database	Requirements imply users, buses, routes, schedules, seats, bookings, payment status, and ticket records.	No database or persistent browser storage found; in-memory JavaScript state only.
External service	QR-code library and CDN-hosted styling/icon/font assets are referenced. No payment, email, or maps API is shown.	Tailwind CDN, Font Awesome CDN, Google Fonts, QRCode.js CDN; no payment provider.

4. UML and System Diagrams

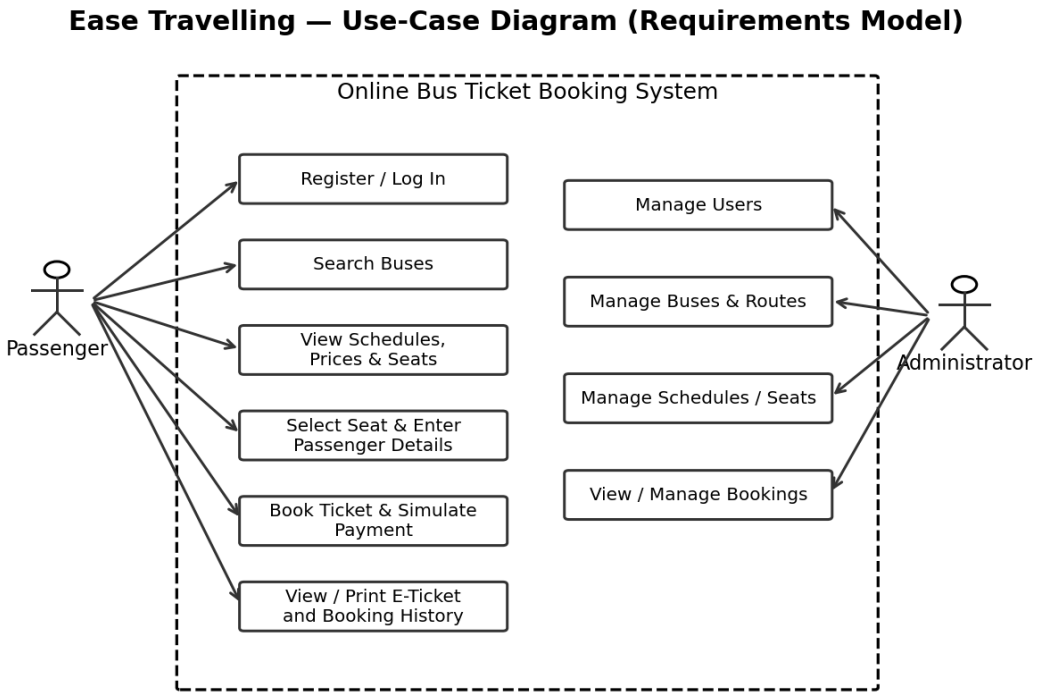


Figure 2. Requirements-based use-case model; administration functions require backend implementation.

4.1 Use-Case Diagram

The use-case model below reflects the requirements document. Passenger and Administrator are the primary actors. Bus operators and booking staff are stakeholders in the requirements, but no separate operator/staff login workflow is established in the supplied HTML.

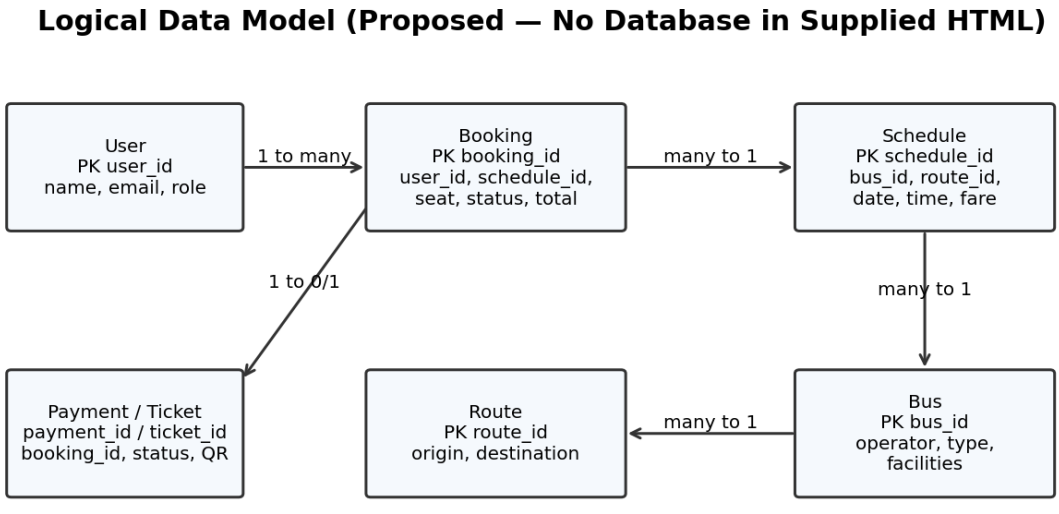


Figure 3. Proposed logical data model; entities are not current database tables.

Suggested passenger use cases: Register/Login, Search Buses, View Bus Details, Select Seats, Book Ticket, Make/Simulate Payment, View/Download Ticket, View Booking History.

Suggested admin use cases: Manage Buses, Routes, Schedules, Bookings, and Users. Keep only use cases supported by your actual project.

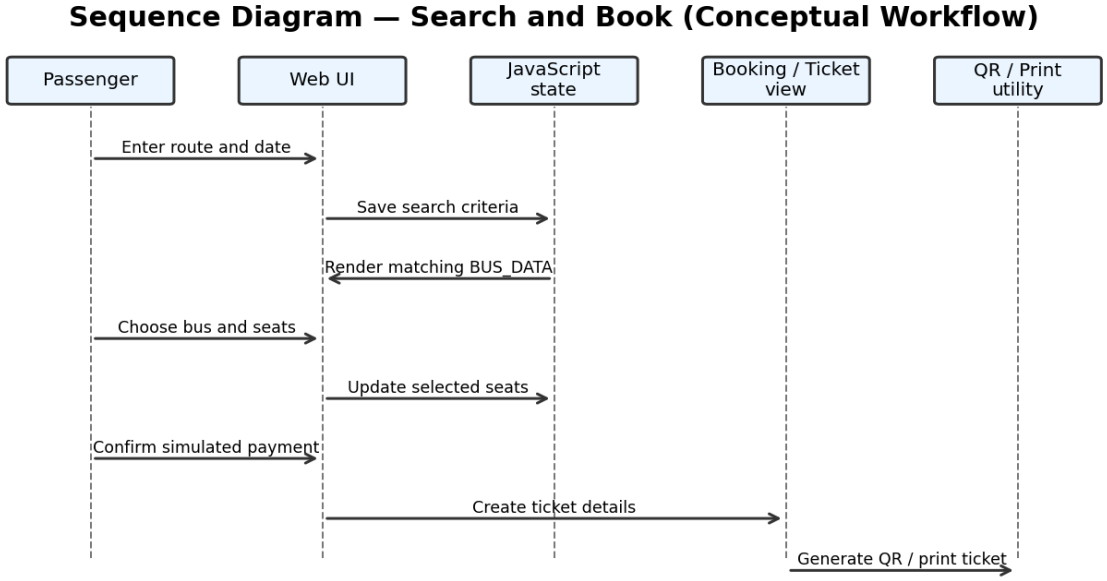


Figure 4. Conceptual sequence for searching and completing a booking.

Activity flow (text): Start → Enter origin, destination, and date → Show sample bus results → Select bus → Select available seat(s) → Enter passenger details → Simulate payment → Generate ticket ID and QR code → Display / print e-ticket → End. If no result or invalid details are provided, return to the relevant form.

4.2 Class Diagram / Data Model

The data model below is a proposed logical model derived from the requirements. It is not a database schema currently implemented by the supplied HTML, which contains no database connection.

Possible entities: User, Bus, Route, Schedule, Seat, Booking, PassengerDetails, Payment, Ticket. Include only entities/classes present in your design.

For each class/entity, document its key attributes, methods (for class diagrams), relationships, and multiplicities where relevant.

4.3 Sequence Diagram

The sequence diagram is conceptual and follows the intended user journey. In the current prototype, steps are performed in browser JavaScript using sample data and temporary state rather than a server or database.

Example flow to adapt: Passenger → Search/Booking Page → Application Logic → Database → Results/Booking Confirmation.

4.4 Activity Diagram (Optional / If Required)

Optional activity view: the passenger searches for a route and date, reviews buses, selects an available seat, enters passenger details, completes the simulated payment step, and receives an e-ticket. If no buses match or required information is invalid, the user should return to search or correct the form.

5. Database Design

5.1 Database Overview

The supplied HTML stores its demo state in JavaScript variables (including currentState and BUS_DATA) while the page is open. No persistent database or browser storage was found. The entities below are therefore a recommended logical design for a future backend, not existing tables.

5.2 Tables / Entities

Table / entity	Primary key	Important fields	Relationships
User	user_id (proposed)	name, email, password hash, role	One user may have many bookings.
Bus	bus_id (proposed)	operator, bus type, facilities	One bus may have many schedules.
Route	route_id (proposed)	origin, destination	One route may be referenced by many schedules.
Schedule	schedule_id (proposed)	bus_id, route_id, date, departure, arrival, fare	Belongs to one bus and one route; may have many bookings.
Booking	booking_id (proposed)	user_id, schedule_id, seat(s), status, total	Belongs to a user and schedule; linked to payment/ticket details.
Payment / Ticket	payment_id / ticket_id (proposed)	booking_id, payment status, QR/ticket data	Associated with a booking; payment remains simulated.

Replace these illustrative rows with your real schema; remove anything not used in the project.

6. User Interface Design

6.1 Design Principles

- Consistent colors, typography, spacing, and navigation.
- Clear labels and readable bus, route, fare, and seat information.

- Responsive design for different screen sizes.
- Helpful validation and confirmation messages.

6.2 Wireframes / Screenshots

Home / Bus Search: A home view with a route/date search form and quick route selection. Search stores criteria in currentState and opens the bus-results view.

Login / Registration: Login and registration forms are present. In the supplied prototype, handlers assign demo user values instead of verifying credentials or creating persistent accounts.

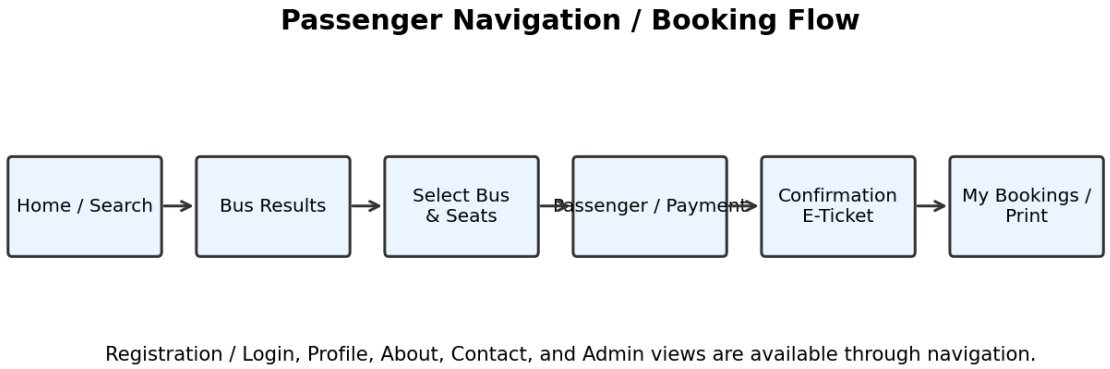


Figure 5. Passenger navigation and booking flow.

Search Results / Bus Details: Bus cards are rendered from sample BUS_DATA and show route/operator details and fare-related information. Live operator data is not connected.

Seat Selection: Seat buttons have available, selected, and booked visual states. The prototype updates selectedSeats in client-side state; it does not lock seats across multiple browsers.

Passenger Details / Payment: The booking flow includes passenger details and a simulated payment step. No real payment provider is connected.

Booking Confirmation / E-Ticket: The confirmation view populates a ticket ID, operator, route, seats, amount, passenger list, and QR code. Print CSS targets the printable ticket.

My Bookings: A booking-list view is present and renders records from the in-memory userBookings state. Persistent history and cross-session access are not demonstrated.

Admin Dashboard: Admin navigation and tab styling are present. The supplied code does not demonstrate a secured backend or persistent administrative changes.

6.3 Navigation Flow

The navigation diagram summarizes the passenger flow from searching to ticket display. The HTML also contains navigation views for Home, Buses, Login, Register, Passenger/Payment, Confirmation, My Bookings, Profile, About, Contact, and Admin.

7. Security and Design Considerations

- Authentication and role-based access: Login/register handlers currently set demo user objects in JavaScript. Server-side identity verification and authorization are not present in the supplied HTML. Admin access must be protected before production use.

- Password handling: The supplied front-end code does not show secure password storage or server-side account management. A future backend must store salted password hashes and never store plaintext passwords.
- Input validation: The interface collects route/date and passenger information, but server-side validation is not available because no backend is connected. Validate all required fields, dates, and passenger inputs in both client and server layers.
- Booking/seat consistency: Seat states are represented in the UI, but no server-side locking or transaction control is shown. A future database transaction or reservation lock should prevent two users from booking the same seat.
- Payment safety: Payment is simulated only. No bank credentials, card processing, or real payment gateway should be used with this prototype.
- Privacy: The requirements imply passenger account and booking details. The prototype includes demo passenger/ticket values; production deployment should minimize collected data, protect personal information in transit and at rest, and restrict access to authorized roles.

8. Design Decisions and Limitations

Decision / limitation	Reason	Impact
Single-page client-side prototype	The supplied website is one HTML file with JavaScript view navigation.	Easy to demonstrate, but business logic and data are not separated into server components.
Sample data and temporary state	No API, database, localStorage, or sessionStorage was found.	Data may reset when the page reloads; users cannot rely on durable records.
Simulated payment	The requirements specify simulated payment and exclude real gateway integration.	Safe for a classroom demo; not suitable for accepting real payments.
Responsive blue/navy visual theme	HTML defines navy, sky blue, cyan, pale blue background, Inter font, and responsive Tailwind utilities.	Consistent, readable interface for desktop and mobile layouts; actual device testing remains necessary.